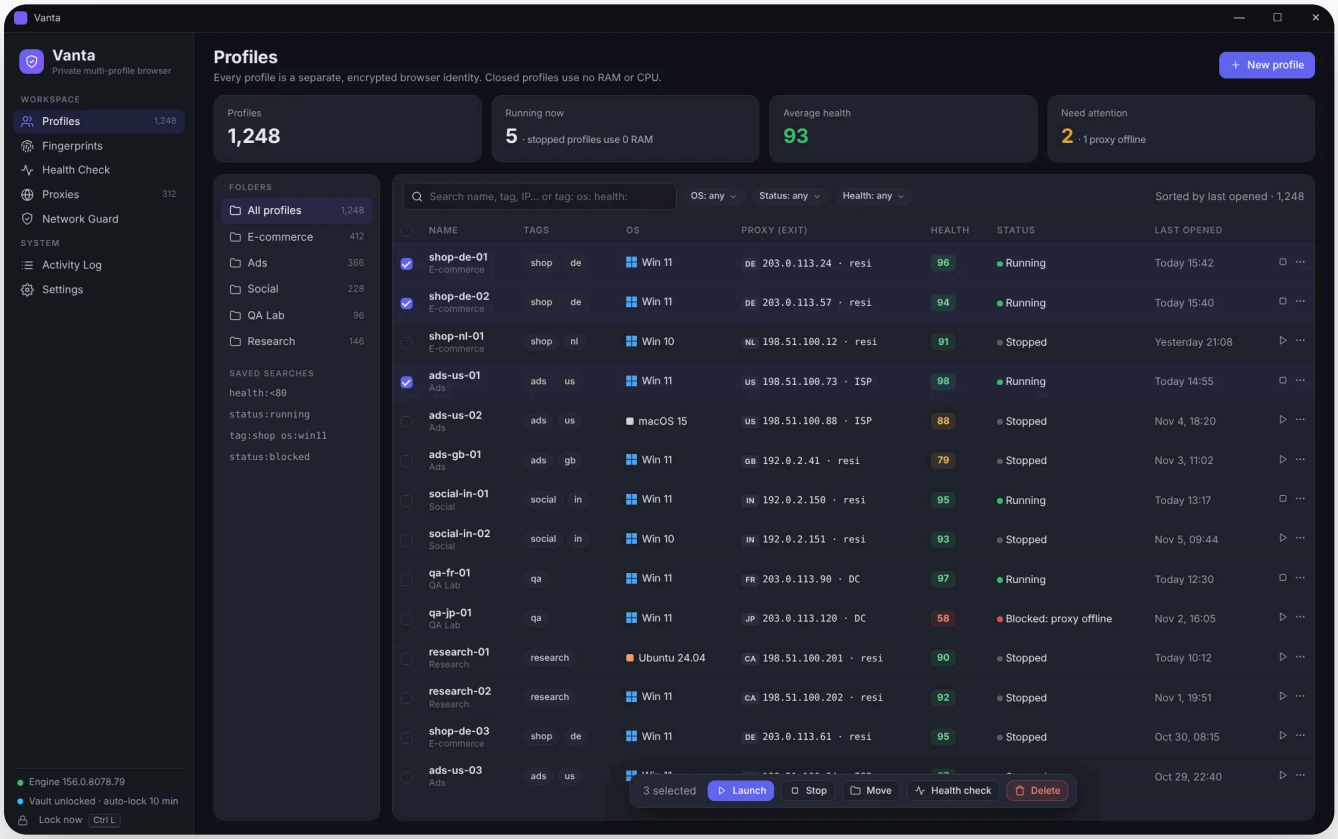
Master Plan v3 · ৬ নভেম্বর ২০২৬ · সরল সংস্করণ

Vanta — মাস্টার প্ল্যান ও UI ডিজাইন (v3)

Windows 10/11-এর জন্য নিজের ইঞ্জিনসহ, লোকাল-অনলি, হালকা ও দ্রুত মাল্টি-প্রোফাইল প্রাইভেসি ব্রাউজার।
৮টা ফিচার, ১০টা স্ক্রিন, নিজের Chromium ইঞ্জিন (ungoogled-chromium-windows 156.0.8078.79-1.1 + Vanta প্যাচ)।

v2 থেকে বদল: আপনার কথামতো ১৩টা ফিচার বাদ, বাকি ৮টা নতুন করে F01–F08 নম্বর, স্ক্রিন ২২ থেকে ১০, ইঞ্জিনের প্যাচ দল ৯ থেকে ৮, Rust ক্রেট ১১ থেকে ৮। মূল নীতি: যত সরল, তত দ্রুত, তত হালকা, তত কম ভুল।



০. এই নথি কীভাবে ব্যবহার হবে

এটাই এখন থেকে Vanta বানানোর একমাত্র নির্দেশিকা; Master Plan v2-এর জায়গা নিল। কোনো সিদ্ধান্ত বদলালে আগে এই নথি বদলাবে, তারপর কোড। পুরোনো নথির সাথে অমিল হলে এই নথি জিতবে।

প্রতিটা ফিচারের পাতায় আছে: **কী করে**, **কীভাবে কাজ করে**, আর **কখন "শেষ" বলব** (মাপা যায় এমন পরীক্ষা)। পরীক্ষা পাস না করলে ফিচার "শেষ" নয়।

> স্ক্রিনের ছবিগুলো আসল HTML/CSS কোড দিয়ে বানিয়ে Chromium-এ রেন্ডার করা, AI-আঁকা নয়; এই একই ডিজাইন-সিস্টেম আসল অ্যাপের UI-তে ব্যবহার হবে। সব স্ক্রিনে একই নমুনা-ডাটা (একই প্রোফাইল, IP, সময়)। IP ঠিকানাগুলো RFC 5737-এর নমুনা (203.0.113.x, 198.51.100.x, 192.0.2.x)। ফিঙ্গারপ্রিন্ট স্ক্রিনের মানগুলো আমাদের ডিভাইস ডাটাবেসের আসল রেকর্ড win11-rtx4070-1440p থেকে নেওয়া।

১. পণ্যের সংজ্ঞা

Vanta হলো একজন ব্যবহারকারীর জন্য, শুধু নিজের কম্পিউটারে চলা, মাল্টি-প্রোফাইল প্রাইভেসি ব্রাউজার। লক্ষ্য **Windows 10/11 x64**।

পাঁচটা মূল প্রতিশ্রুতি

1. প্রতিটা প্রোফাইল একটা সঙ্গতিপূর্ণ আসল ডেস্কটপ কম্পিউটারের মতো দেখায়।
2. প্রোফাইলগুলো একে অপরকে দেখতে বা ছুঁতে পারে না।
3. প্রোফাইলের প্রক্সির বাইরে দিয়ে কোনো ট্রাফিক যায় না (কিল সুইচ)।
4. সরল, হালকা, দ্রুত (অংশ ৮-এর বাজেট)।
5. কারো কাছে কিছু পাঠানো হয় না: টেলিমেট্রি নেই, ক্লাউড নেই।

১.১ যা কখনো বানানো হবে না

বিষয়	কারণ
পাবলিক API / SDK / REST / MCP সার্ভার	লোকাল-অনলি নীতি
টিম, রোল, শেয়ারিং	একজন ব্যবহারকারী
ক্লাউড সিঙ্ক, ক্লাউড ব্রাউজার/ফোন, ক্লাউড ব্যাকআপ	কিছুই আপনার PC ছাড়ে না
যেকোনো টেলিমেট্রি	CI-তে no-telemetry পরীক্ষা
নির্দিষ্ট anti-bot কোম্পানিকে টার্গেট/বাইপাস, CAPTCHA সমাধান, অ্যাকাউন্ট ফার্মিং/শপ, SMS-ভেরিফিকেশন	নীতিগত সীমা
অটোমেশন, মোবাইল ও iOS পারসোনা, এবং অংশ ২-এর ১৩টা বাদ দেওয়া ফিচার	সরলতা (আপনার সিদ্ধান্ত, ৬ নভেম্বর ২০২৬)

১.২ বাধ্যতামূলক নিয়ম

#	নিয়ম	কোথায়
R1	Windows মূল লক্ষ্য	CI, লঞ্চার, প্যাকেজিং
R2	৮টা ফিচার F01–F08, এর বেশি নয়; নতুন ফিচার যোগ হবে শুধু এই নথি বদলে	অংশ ৩
R3	ইঞ্জিন = নিজের: ungoogled-chromium-windows + Vanta প্যাচ, GitHub Actions-এ build	অংশ ৫
R4	ভিত্তি ভার্সন ungoogled-chromium-windows 156.0.8078.79-1.1 (১ নভেম্বর ২০২৬-এর রিলিজ)	অংশ ৫
R5	ফিঙ্গারপ্রিন্টের Chrome ভার্সন = ইঞ্জিনের নিজের ভার্সন। প্রোফাইলে আলাদা করে রাখা হয় না, প্রতি লঞ্চে ইঞ্জিন থেকে নেওয়া হয়; না মিললে লঞ্চ বন্ধ (নিয়ম C030, ইঞ্জিনে exit 87)	অংশ ৬
R6	হালকা ও দ্রুত	অংশ ৮
R7	পেশাদার মান, দ্রুত-ব্যর্থ পাইপলাইন	অংশ ৫.৪, ৯
R8	GitHub অ্যাকাউন্ট হারালেও কিছু হারাবে না (মিরর, অফলাইন কী)	অংশ ৭
R9	আপনার সাথে কথা বাংলায়; কোড ও ডকুমেন্ট ইংরেজিতে	—
R10	সব স্পুফিং ইঞ্জিনের ভেতরে (C++), JavaScript ইনজেকশন নয়	অংশ ৫.৩
R11	র‍্যান্ডম মানের বদলে আসল-ডিভাইস ডাটা; noise প্রতি প্রোফাইলে স্থির (seeded)	F02, F03
R12	সব নিজেরা বানাব; অন্যের exe শুধু রেফারেন্স	অংশ ৫
R13	সরলতা: যে জিনিস Windows বা ব্রাউজার নিজেই দেয় (Task Manager, Snap, কুকি সেটিংস, ফাইল ব্যাকআপ) তা Vanta আবার বানাবে না	সব অংশ

২. কী বাদ গেল, আর তার প্রভাব

প্রতিটা বাদ-পড়া ফিচারের প্রভাব খুঁটিয়ে দেখা হয়েছে, যাতে কোনো ঝুলে থাকা অংশ না থাকে। বাঁ কলামে v2-এর পুরোনো নম্বর।

v2 ID	ফিচার	কী থাকল / কী বদলাল
F06	Workflow Builder	পুরো অটোমেশন বাদ। ফলে ইঞ্জিনে CDP-সংযোগ আর লাগে না: লঞ্চার --remote-debugging-io-pipes দেয় না, প্যাচ দল G9 (অটোমেশন লুকানো) বাদ, vanta-workflow ক্রেট বাদ। পেজ থেকে অটোমেশন ধরা পড়ার ঝুঁকিও কমে।
F07	Synchronizer	বাদ (এটা CDP-নির্ভর ছিল)। প্রতিটা উইন্ডো স্বাভাবিকভাবে হাতে চালাবেন।
F08	Mobile Profiles	Android পারসোনা বাদ। ডিভাইস ডাটাবেস থেকে ২টা Android রেকর্ড সরানো (১৪টা ডেস্কটপ রেকর্ড থাকল); মোবাইল-নিয়ম সরিয়ে নিয়ম C001–C030 নতুন করে সাজানো; প্যাচ দল G3 থেকে মোবাইল অংশ বাদ। কোডে ইতিমধ্যে করা।
F10	Bulk Create Wizard	একটা একটা করে "New profile" (স্ক্রিন ০৩)। এক প্রক্সি-এক প্রোফাইল নিয়ম ফর্মেই থাকে।
F11	Linkability Radar	বাদ। ঝুঁকি কমানো থাকে ডিফল্ট দিয়ে: আলাদা প্রক্সি, আলাদা seed, একই প্রক্সি দিলে সতর্কতা। নিয়ম C025 শুধু রেকর্ডটা কতটা সাধারণ তা দেখায়।
F12	Engine Update & Persona Aging	"Age personas" দরকারই থাকছে না: প্রোফাইলে Chrome ভার্সন রাখা হয় না, প্রতি লঞ্চে ইনস্টল করা ইঞ্জিন থেকে বসে (R5 বজায়, নিয়ম C030 + ইঞ্জিনের exit 87 দুই স্তরে)। ইঞ্জিন একটাই ভার্সন, অ্যাপের সাথে একই ইনস্টলারে আসে; অ্যাপের ভেতরে আপডেটার, রোলব্যাক, side-by-side ইঞ্জিন বাদ। সেটিংসে শুধু ভার্সন দেখা যায়।
F13	Snapshots & Backup	বাদ। .vxpack ফরম্যাট বাদ। ডাটা ফোল্ডার এনক্রিপ্টেড, তাই চাইলে Windows-এর সাধারণ ফাইল ব্যাকআপ দিয়ে %LOCALAPPDATA%\Vanta কপি রাখা যায় (পাসওয়ার্ড বা রিকভারি কী ছাড়া খোলা যায় না)।
F14	Cookie & Storage Editor	বাদ। কুকি দেখা/মোছা ব্রাউজারের নিজের সেটিংস থেকেই করা যায়।
F15	Extension Library	বাদ। প্রয়োজনে প্রতিটা প্রোফাইলের ভেতরে সাধারণভাবে এক্সটেনশন বসানো যায়; Vanta আলাদা লাইব্রেরি রাখে না। vanta-browserdata ক্রেট বাদ।
F16	Command Palette (Ctrl+K)	বাদ। প্রোফাইল টেবিলের সার্চ বক্সে একই সার্চ ভাষা থাকে।
F17	Resource & Traffic Monitor	বাদ। অটো-স্লিপ, প্রতি-প্রোফাইল RAM/CPU সীমা, ট্রাফিক-মিটার ও ট্রাফিক-ক্যাপ বাদ। Job Object থাকে শুধু একটা কাজে: অ্যাপ বন্ধ হলে সব প্রোফাইল-প্রসেস বন্ধ। ব্যবহার দেখতে Windows Task Manager।
F18	Panic Button & Secure Delete	প্যানিক হটকি, উইন্ডো লুকানো, ফোল্ডার-মোছা বাদ। থাকে: অটো-লক আর Ctrl+L (F06)। সাধারণ Delete-এ প্রোফাইলের কী মুছে যায়, তাই ফাইল অপঠনযোগ্য হয় (কোনো বাড়তি কোড নয়)।
F19	Window Workspaces	বাদ। উইন্ডো Windows-এর নিজের Snap দিয়ে সাজাবেন।

যা সরল হলো: লঞ্চারে CDP নেই, ইঞ্জিন-আপডেটার নেই, স্ন্যাপশট ফরম্যাট নেই, অটোমেশন রানার নেই, মোবাইল পারসোনা নেই। ফলে কম কোড, কম প্রসেস, কম RAM, ভুলের কম সুযোগ, আর পেজের চোখে ধরা পড়ার মতো কম চিহ্ন।

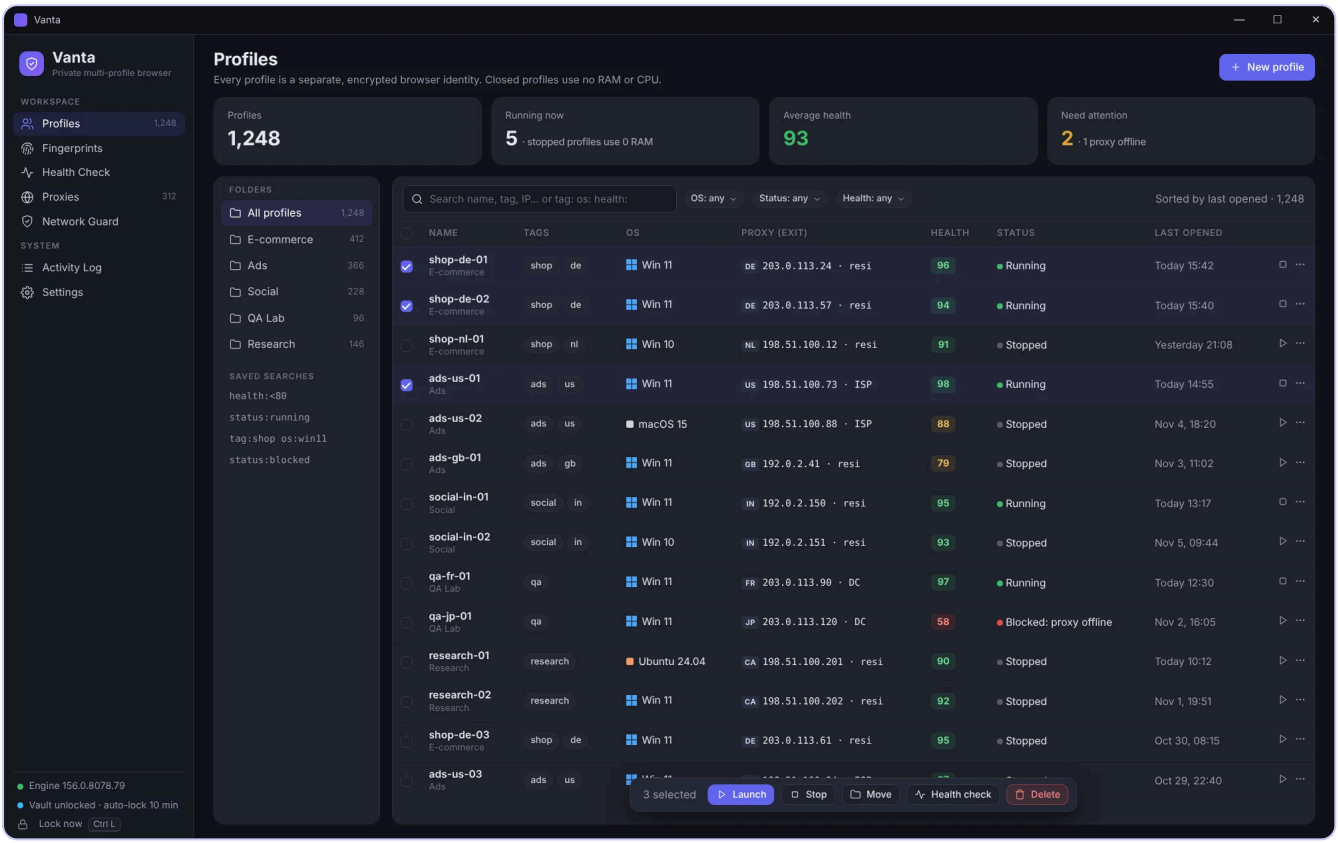
৩. ৮টা ফিচার ও ১০টা স্ক্রিন — এক নজরে

নতুন ID	ফিচার	স্ক্রিন #	v2-তে ছিল
F01	প্রোফাইল ম্যানেজার Profile Manager	02	F01 + F10/F16-এর ছোট অংশ
F02	ফিঙ্গারপ্রিন্ট স্টুডিও Fingerprint Studio	04	F02
F03	হেলথ চেক (লোকাল টেস্ট ল্যাব) Health Check	05	F03
F04	প্রক্সি হাব Proxy Hub	06	F04
F05	নেটওয়ার্ক গার্ড Network Guard	07	F05
F06	ভল্ট লক Vault Lock	01	F09
F07	জিও অটো-ম্যাচ Geo Auto-Match	08	F20
F08	লোকাল অ্যাক্টিভিটি লগ Activity Log	09	F21
—	নতুন প্রোফাইল ফর্ম (F01-এর অংশ)	03	F10-এর বদলে
—	সেটিংস	10	v2 স্ক্রিন 20, 21, 22 মিলিয়ে

ডিজাইন-সিস্টেম

- Windows 11 টাইটেল বার (minimize, maximize, close)।
- বাঁ দিকে ছোট মেনু, ২টা দলে: **Workspace** (Profiles, Fingerprints, Health Check, Proxies, Network Guard) আর **System** (Activity Log, Settings)। নিচে: ইঞ্জিন ভার্সন, ভল্টের অবস্থা, Lock now (Ctrl+L)।
- ডার্ক থিম, মূল রং ইন্ডিগো #6366f1; সবুজ = ঠিক, হলুদ = নজর, লাল = বন্ধ। ফন্ট Inter (Windows-এ Segoe UI fallback)।

F01 প্রোফাইল ম্যানেজার · Profile Manager · v2-তে F01 + F10/F16-এর ছোট অংশ

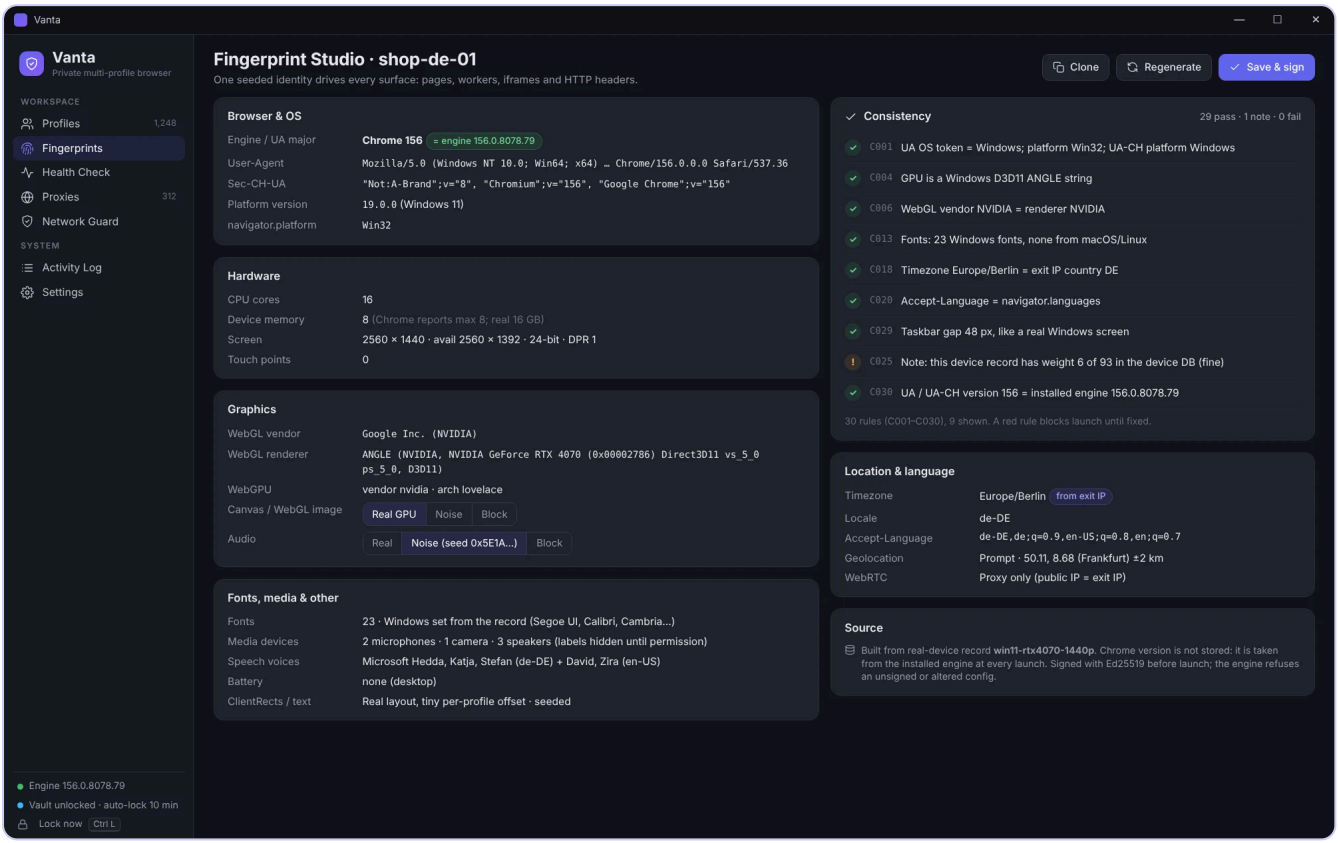


কী করে: সব প্রোফাইল এক টেবিলে। ফোল্ডার, ট্যাগ, সেভ করা সার্চ। একটা একটা করে নতুন প্রোফাইল বানানো (স্ক্রিন ০৩), আর একসাথে কয়েকটা বেছে Launch / Stop / Move / Health check / Delete। প্রতিটা প্রোফাইল আলাদা, এনক্রিপ্টেড ব্রাউজার-পরিচয়। বন্ধ প্রোফাইল RAM বা CPU নেয় না।

কীভাবে কাজ করে: vanta-store: SQLite + FTS5 ইনডেক্স (নাম, ট্যাগ, IP, নোট, ফোল্ডার)। ভার্চুয়াল টেবিল (শুধু দেখা যাওয়া সারি রেন্ডার)। সার্চ ভাষা: tag: os: status: health: proxy: folder: আর OR। স্ট্যাটাস আসে vanta-launcher-এর আসল প্রসেস অবস্থা থেকে। Delete করলে প্রোফাইলের নিজের কী-ও মুছে যায়, তাই তার ফাইল আর পড়া যায় না (আলাদা ফিচার ছাড়াই)।

কখন "শেষ" বলব: ১০,০০০ প্রোফাইলে সার্চ ৫০ ms-এর কম; নতুন প্রোফাইল ১ সেকেন্ডের কম; প্রসেস মেরে দিলে ২ সেকেন্ডে Stopped দেখায়। (সার্চ ও ডাটাবেস অংশ ইতিমধ্যে লেখা ও টেস্ট পাস।)

F02 ফিঙ্গারপ্রিন্ট স্টুডিও · Fingerprint Studio · v2-তে F02

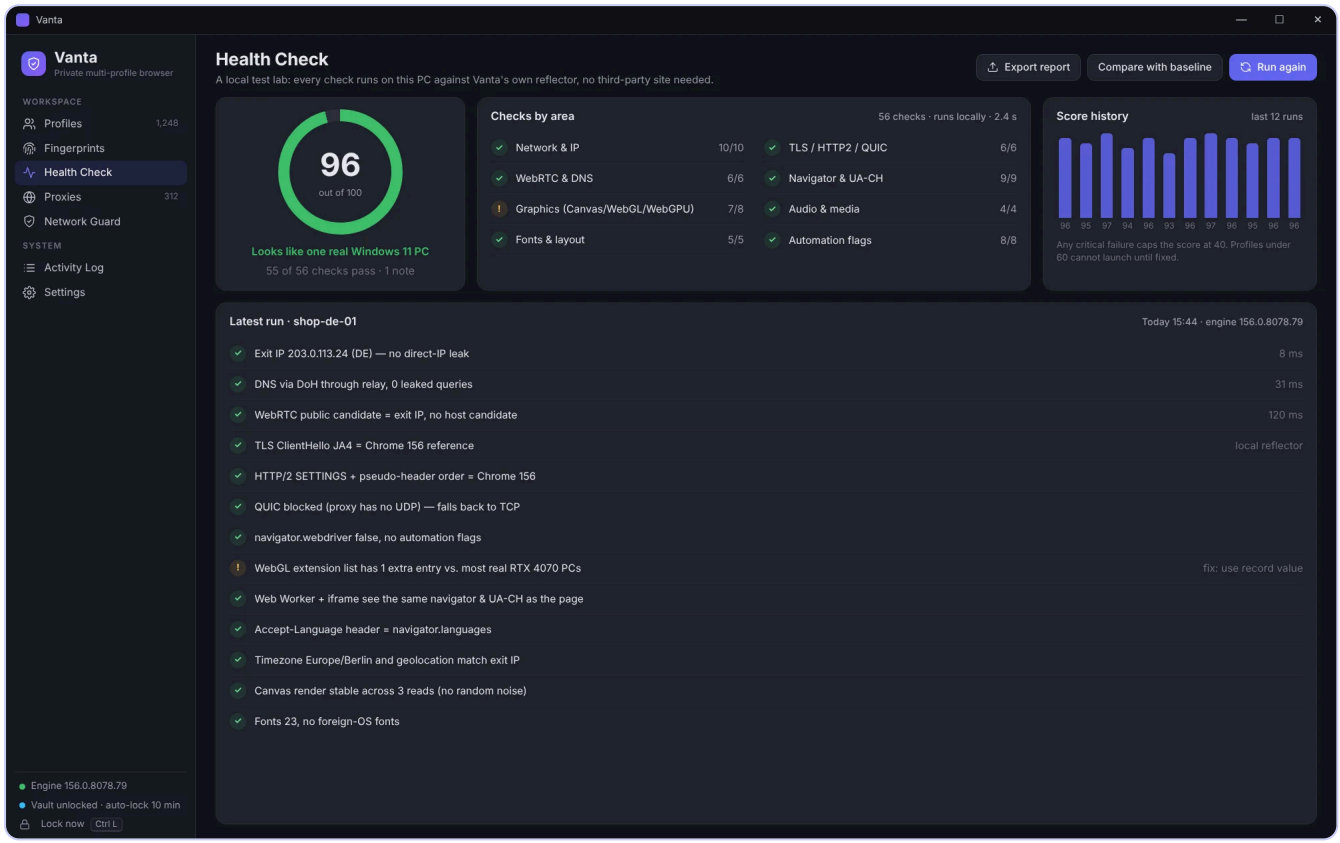


কী করে: প্রতিটা প্রোফাইলের ডিভাইস-পরিচয় দেখা ও এডিট: ব্রাউজার ও OS, হার্ডওয়্যার, গ্রাফিক্স, ফন্ট, মিডিয়া, ভাষা ও লোকেশন। ৩০টা সঙ্গতি-নিয়ম (C001–C030) সাথে সাথে পরীক্ষা হয়; লাল নিয়ম থাকলে লঞ্চ হয় না। শুধু ডেস্কটপ পরিচয় (Windows 10/11, macOS, Linux)।

কীভাবে কাজ করে: vanta-fingerprint: আসল-ডিভাইস রেকর্ড থেকে পরিচয় (র‍্যান্ডম নয়, seed দিয়ে বাছাই)। এক seed থেকে সব সারফেস: পেজ, Worker, iframe, HTTP হেডার। Canvas/WebGL/Audio-তে Real / Noise / Block; Noise প্রতি প্রোফাইলে স্থির। Chrome ভার্সন প্রোফাইলে আলাদা করে রাখা হয় না, প্রতি লঞ্চে ইনস্টল করা ইঞ্জিন থেকে নেওয়া হয় (R5)। কনফিগ Ed25519 দিয়ে সাইন; ইঞ্জিন সাইন ছাড়া কনফিগ মানে না।

কখন "শেষ" বলব: CreepJS-এ "lies" শূন্য; একই প্রোফাইল ১০ বার খুললে ফিঙ্গারপ্রিন্ট হ্যাশ একই; Worker আর পেজের navigator হুবহু এক। (নিয়ম, UA/UA-CH ও সাইন অংশ ইতিমধ্যে লেখা ও টেস্ট পাস।)

F03 হেলথ চেক (লোকাল টেস্ট ল্যাব) · Health Check · v2-তে F03

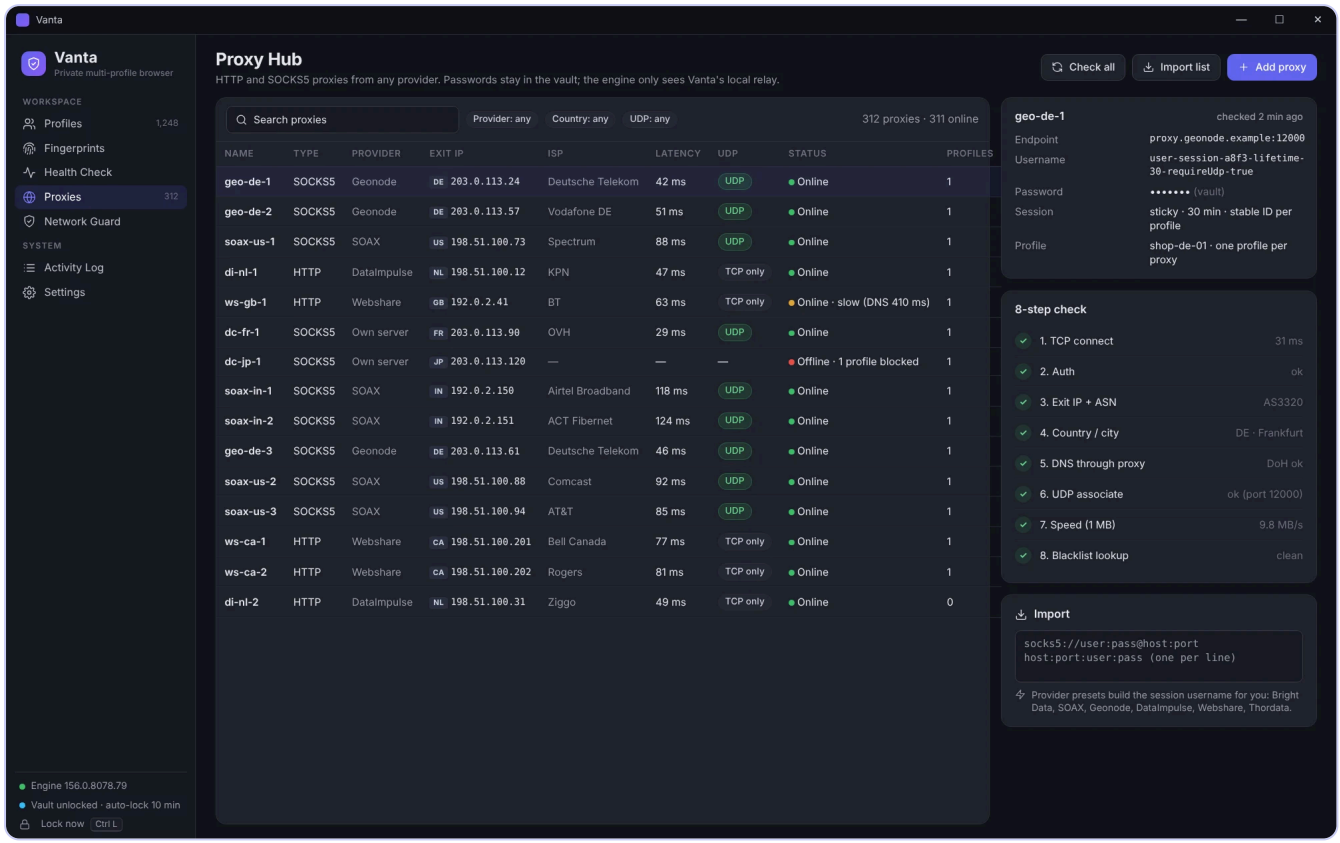


কী করে: ৮টা এলাকায় ৫৬টা চেক, ০–১০০ স্কোর, আর কী ঠিক করতে হবে তার নির্দেশ। সব চেক আপনার PC-তেই চলে, Vanta-র নিজের রিফ্লেক্টর দিয়ে।

কীভাবে কাজ করে: vanta-health: লোকাল HTTPS রিফ্লেক্টর (TLS ClientHello/JA4, HTTP/2 SETTINGS ক্রম, হেডার ক্রম), STUN রেসপন্ডার (WebRTC), DoH লিক টেস্ট, পেজ-সাইড প্রোব (navigator, UA-CH, Canvas স্থিরতা, Worker/iframe মিল)। প্রোব-পেজ ফল পাঠায় লোকাল রিফ্লেক্টরে, তাই ব্রাউজারে কোনো ডিবাগ-সংযোগ (CDP) লাগে না। কোনো critical ফেল হলে স্কোর সর্বোচ্চ ৪০; ৬০-এর নিচে লঞ্চ বন্ধ।

কখন "শেষ" বলব: জেনে-শুনে ভাঙা ১০টা প্রোফাইলে প্রতিটা ভুল ধরা পড়ে; ভালো প্রোফাইলে ৯০+; পুরো চেক ৩ সেকেন্ডের কম।

F04 প্রক্সি হাব · Proxy Hub · v2-তে F04

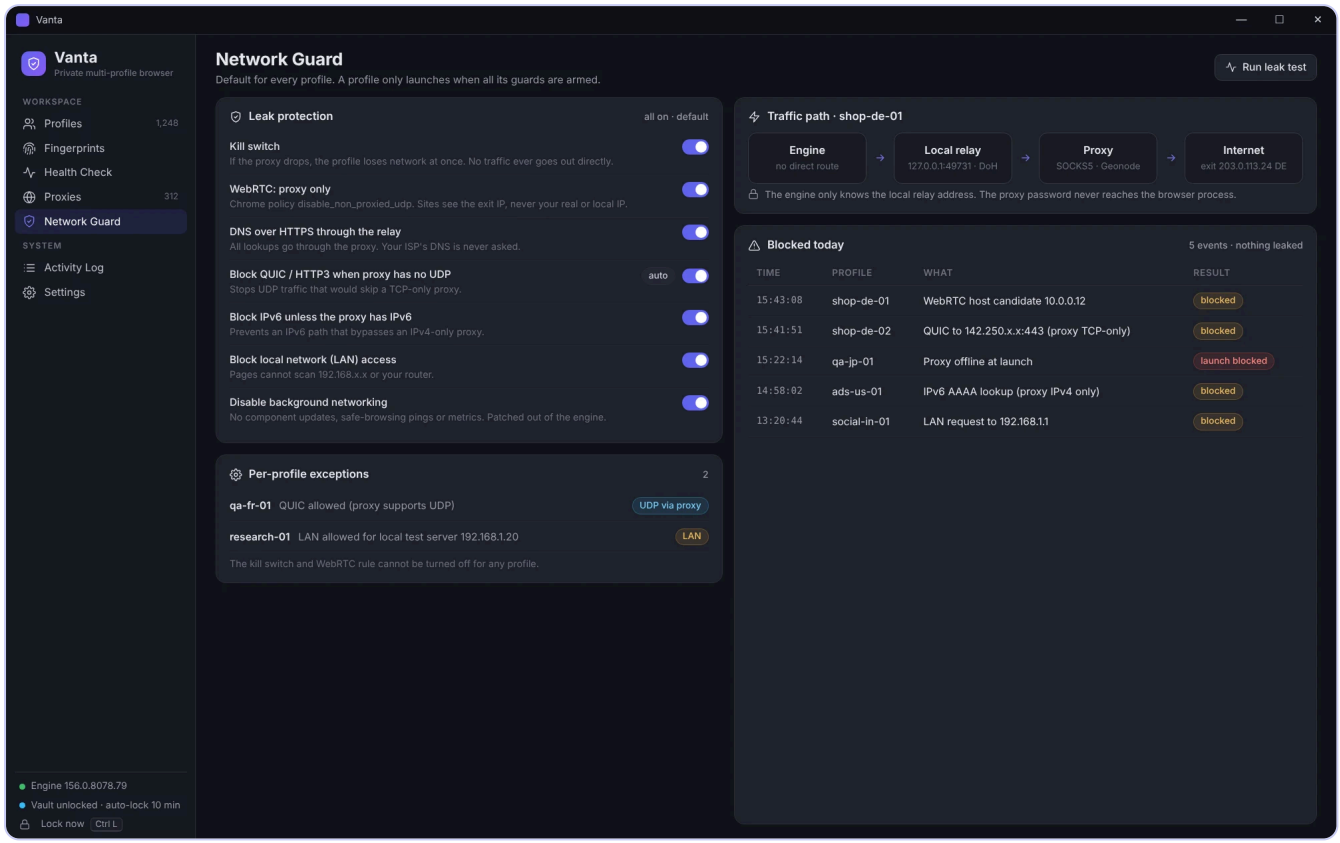


কী করে: যেকোনো প্রোভাইডারের HTTP ও SOCKS5 প্রক্সি একবারে ইমপোর্ট, ৮ ধাপে চেক (সংযোগ, অথ, exit IP ও ASN, দেশ/শহর, DNS, UDP, গতি, ব্ল্যাকলিস্ট)। পাসওয়ার্ড ভল্টে থাকে। প্রোভাইডার প্রিসেট নিজেই সেশন-ইউজারনেম বানায়। ডিফল্ট: এক প্রক্সি এক প্রোফাইল।

কীভাবে কাজ করে: vanta-proxy: ৬টা প্রোভাইডার গ্রামার (Bright Data, SOAX, Geonode, DataImpulse, Webshare, Thordata), প্রতি প্রোফাইলে স্থির সেশন-ID, UDP ASSOCIATE প্রোব। অফলাইন প্রক্সির প্রোফাইল লঞ্চ ব্লক। একই প্রক্সি দ্বিতীয় প্রোফাইলে দিলে সতর্কতা।

কখন "শেষ" বলব: ১,০০০ লাইনের তালিকা ২ সেকেন্ডে পার্স; ভুল লাইন আলাদা করে দেখায়; পাসওয়ার্ড কখনো লগ বা UI-তে খোলা দেখা যায় না।

F05 নেটওয়ার্ক গার্ড · Network Guard · v2-তে F05

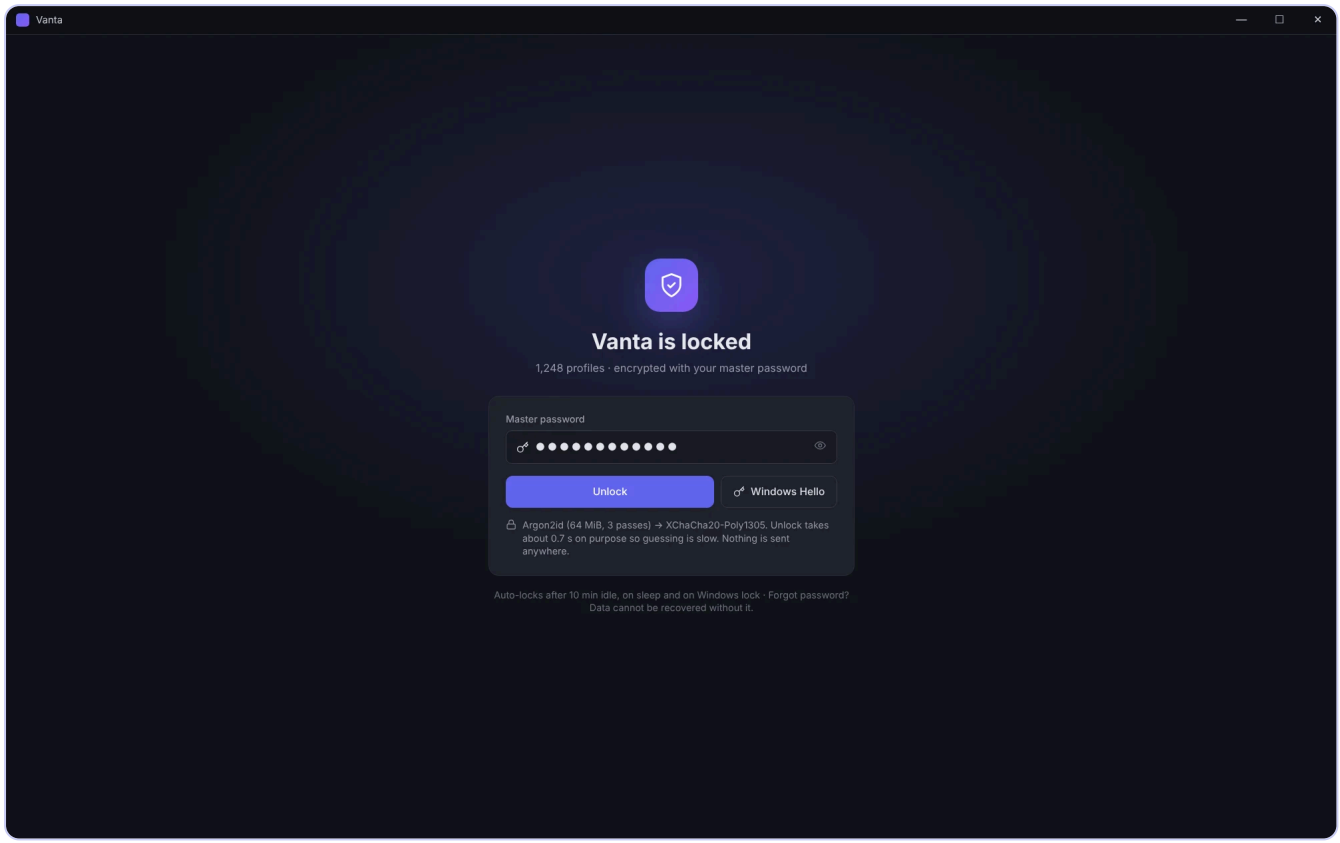


কী করে: কিল সুইচ, WebRTC শুধু প্রক্সি দিয়ে, DNS-over-HTTPS, প্রক্সিতে UDP না থাকলে QUIC বন্ধ, IPv6 ও LAN ব্লক, ব্যাকগ্রাউন্ড নেটওয়ার্ক বন্ধ। সব গার্ড চালু না হলে প্রোফাইল খোলে না।

কীভাবে কাজ করে: ইঞ্জিন শুধু 127.0.0.1-এর লোকাল রিলে চেনে (Rust, tokio)। রিলে প্রক্সির সাথে কথা বলে আর DoH করে। প্রক্সি পড়ে গেলে রিলে সংযোগ কেটে দেয়। WebRTC পলিসি disable_non_proxied_udp ইঞ্জিন-প্যাচে বাধ্যতামূলক। কিল সুইচ আর WebRTC নিয়ম কোনো প্রোফাইলে বন্ধ করা যায় না।

কখন "শেষ" বলব: প্রক্সি বন্ধ করে দিলে ১ সেকেন্ডের মধ্যে প্রোফাইলের নেট বন্ধ; লোকাল লিক-টেস্টে আসল IP কোথাও দেখা যায় না।

F06 ভল্ট লক · Vault Lock · v2-তে F09

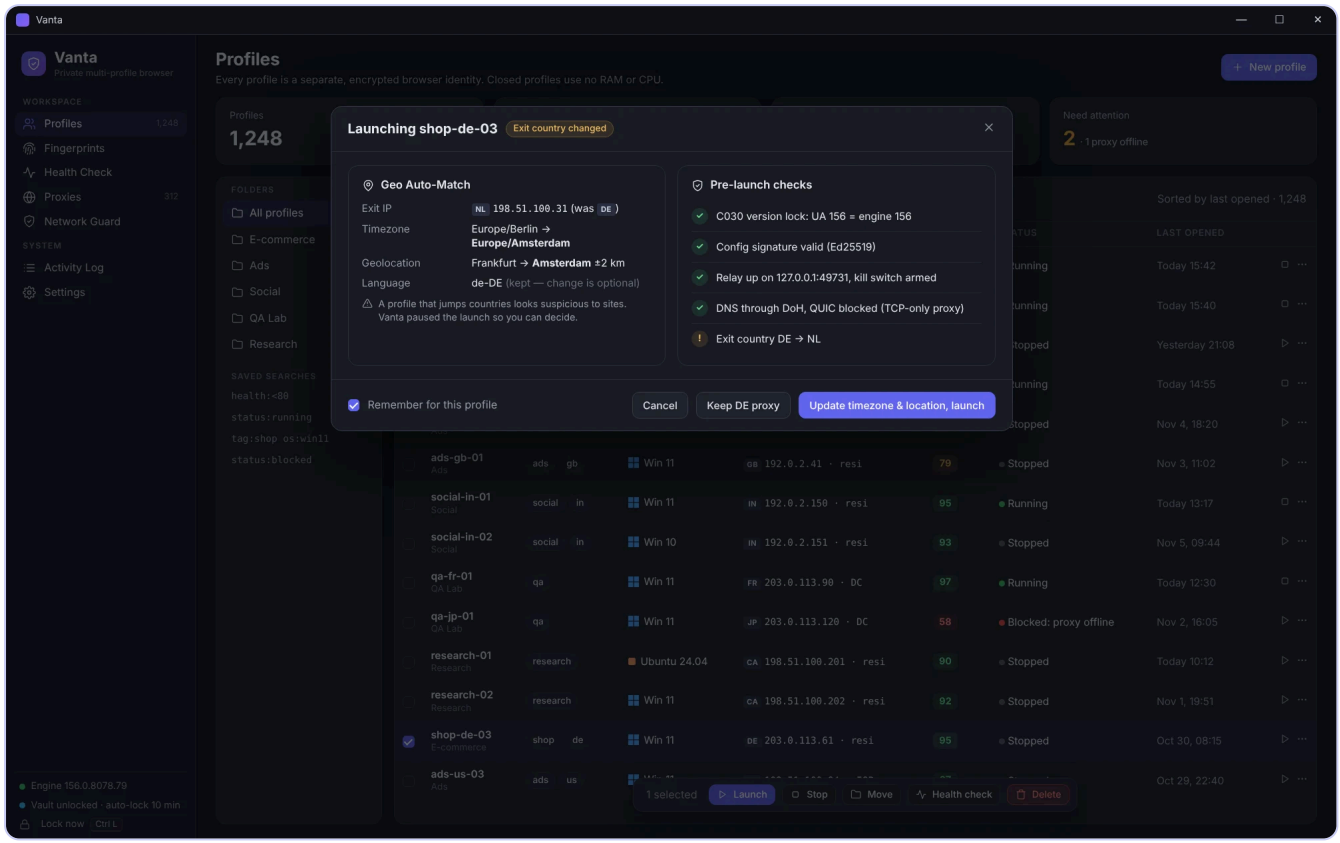


কী করে: মাস্টার পাসওয়ার্ডে সব এনক্রিপ্টেড। Windows Hello দিয়ে খোলা যায়। ১০ মিনিট নিষ্ক্রিয়, স্লিপ, Windows লক বা ল্যাপটপের ঢাকনা বন্ধ হলে নিজে থেকে লক। Ctrl+L দিয়ে সাথে সাথে লক।

কীভাবে কাজ করে: vanta-vault: Argon2id (64 MiB, 3 pass) → পাসওয়ার্ড-কী → র‍্যান্ডম ভল্ট-কী → HKDF → KEK → প্রতি প্রোফাইলে আলাদা র‍্যান্ডম কী (XChaCha20-Poly1305, প্রোফাইল-ID বাঁধা)। পাসওয়ার্ড বদলালে শুধু ভল্ট-কী নতুন করে মোড়ানো হয়। লক হলে কী মেমরি থেকে মুছে যায় (zeroize)। রিকভারি কী একবার প্রিন্ট।

কখন "শেষ" বলব: আনলক ০.৫–১ সেকেন্ড; লক অবস্থায় ডিস্কের কোনো ফাইল পড়ে নাম/কুকি পাওয়া যায় না; স্লিপে গেলেই লক। (ক্রিপ্টো ও অটো-লক অংশ ইতিমধ্যে লেখা ও টেস্ট পাস।)

F07 জিও অটো-ম্যাচ · Geo Auto-Match · v2-তে F20

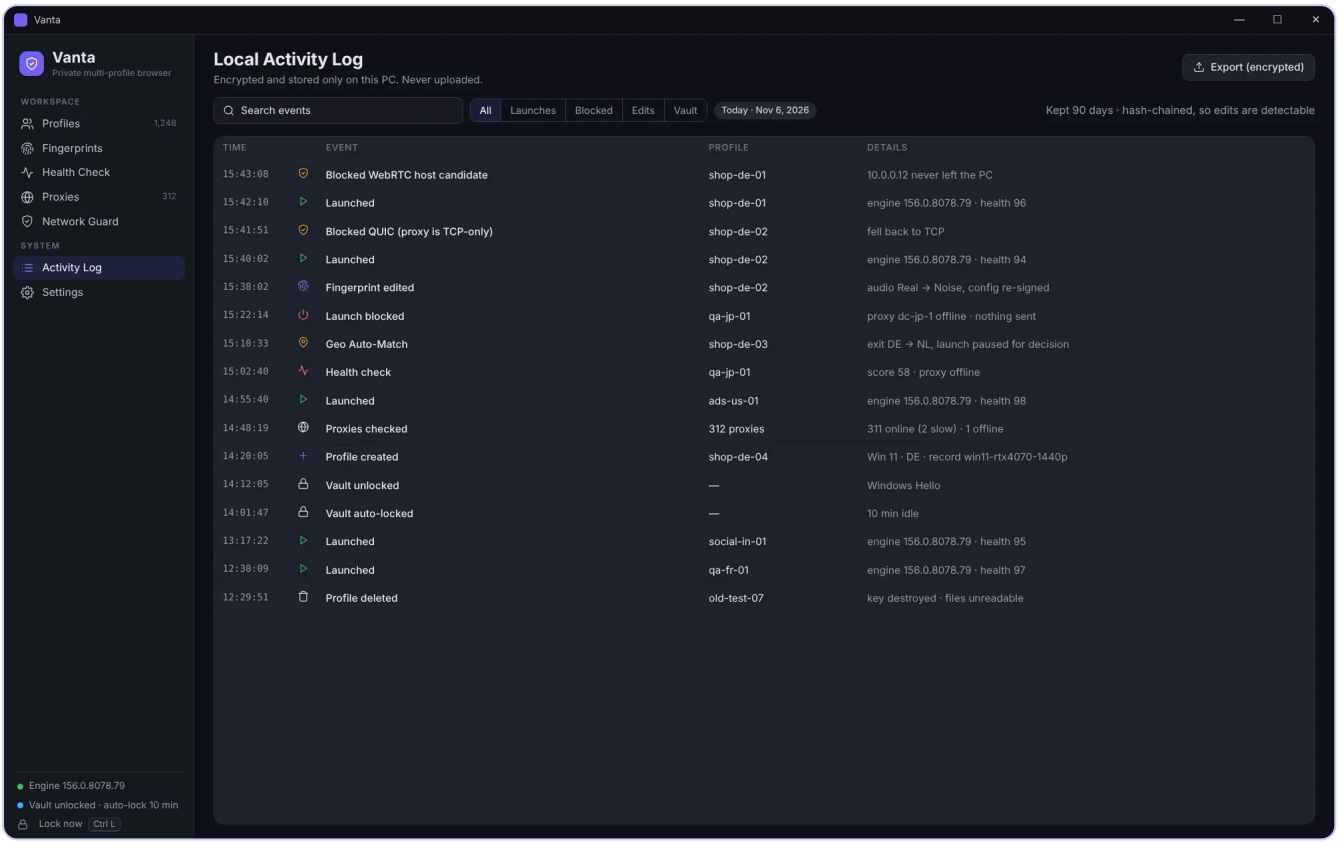


কী করে: প্রক্সির exit IP অনুযায়ী টাইমজোন আর লোকেশন নিজে থেকে মেলানো। দেশ বদলালে লঞ্চ থামিয়ে জিজ্ঞেস করে। ভাষা বদল ঐচ্ছিক, কারণ আসল ভ্রমণকারী নিজের ভাষাই রাখে।

কীভাবে কাজ করে: লঞ্চের আগে রিলে দিয়ে exit IP যাচাই → লোকাল GeoIP ডাটাবেস (অফলাইন) → টাইমজোন/লোকেশন; আগের দেশের সাথে তুলনা। নিয়ম C018 টাইমজোন-দেশ মিল পরীক্ষা করে।

কখন "শেষ" বলব: দেশ বদলালে কখনো চুপচাপ লঞ্চ হয় না; টাইমজোন সবসময় exit IP-র দেশের সাথে মেলে।

F08 লোকাল অ্যাক্টিভিটি লগ · Activity Log · v2-তে F21

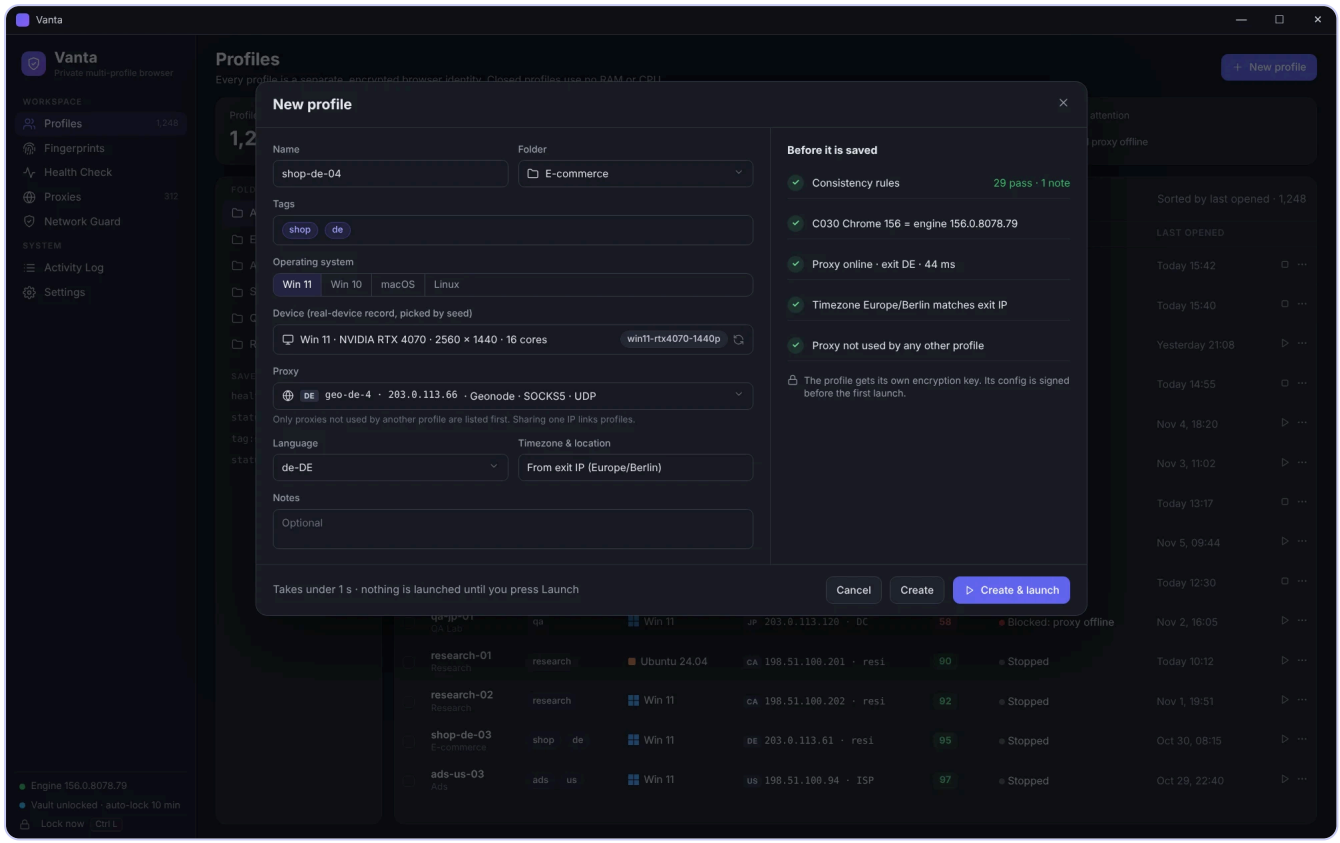


কী করে: কখন কী হলো (লঞ্চ, ব্লক করা লিক, এডিট, প্রোফাইল তৈরি/মোছা, ভল্ট) সব এনক্রিপ্টেড, শুধু আপনার কম্পিউটারে, ৯০ দিন।

কীভাবে কাজ করে: প্রতিটা এন্ট্রি আগেরটার হ্যাশ বহন করে (hash chain), তাই কেউ পুরোনো লাইন বদলালে ধরা পড়ে। ভল্টের আলাদা লগ-কী দিয়ে এনক্রিপ্টেড।

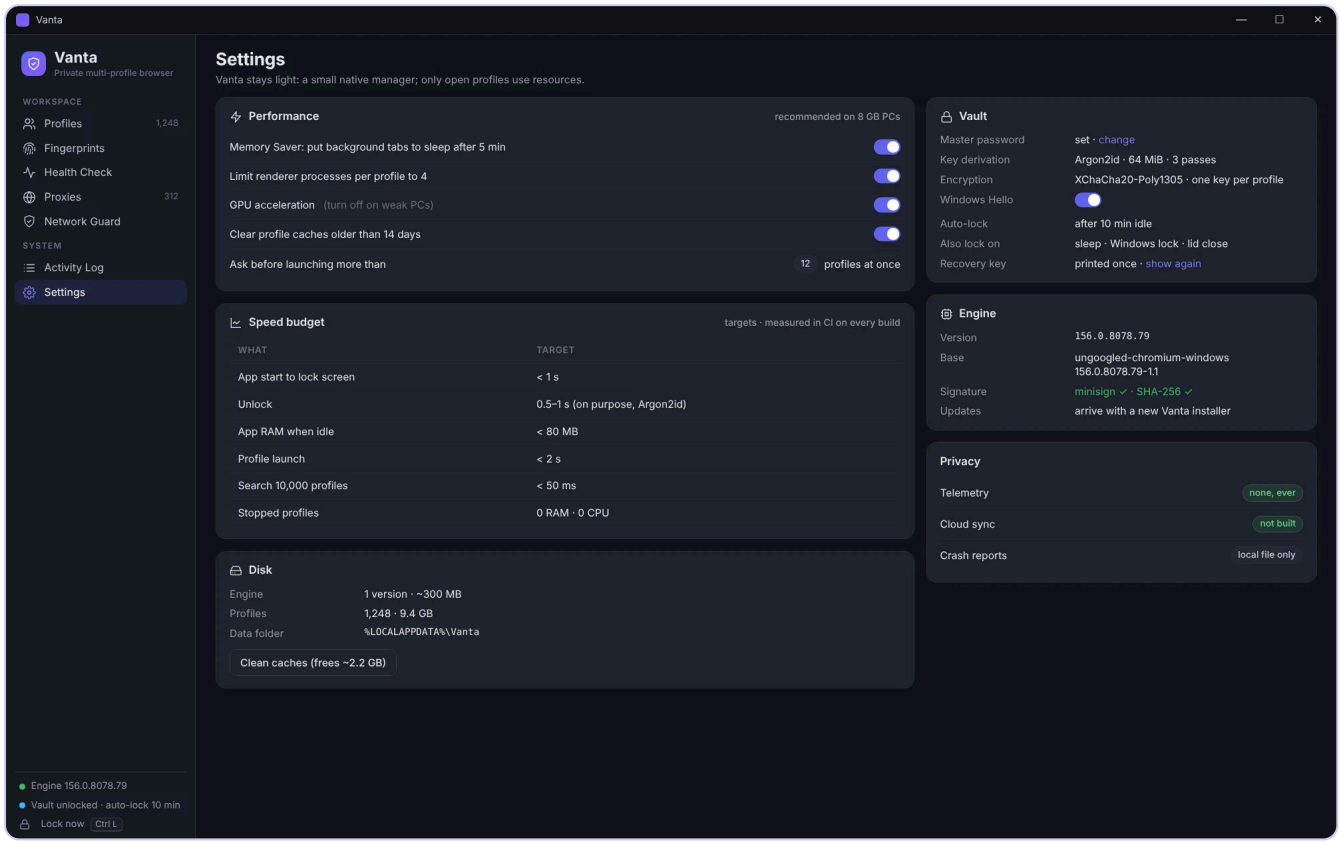
কখন "শেষ" বলব: একটা লাইন বদলালে যাচাই ব্যর্থ হয়; ১০ লাখ এন্ট্রিতেও সার্চ দ্রুত।

স্ক্রিন ০৩ · নতুন প্রোফাইল (F01-এর অংশ)



একটা ফর্মে একটা প্রোফাইল: নাম, ফোল্ডার, ট্যাগ, OS, আসল-ডিভাইস রেকর্ড (seed দিয়ে বাছাই, Regenerate করা যায়), প্রক্সি (অব্যবহৃত প্রক্সি আগে দেখায়), ভাষা। সেভের আগেই নিয়ম, ভার্সন-লক, প্রক্সি আর টাইমজোন পরীক্ষা হয়। বাল্ক উইজার্ড বাদ।

স্ক্রিন ১০ · সেটিংস



পারফরম্যান্স সুইচ (Memory Saver, renderer সীমা, GPU, ক্যাশ পরিষ্কার, একসাথে কতগুলো প্রোফাইল হলে জিজ্ঞেস করবে), গতির বাজেট, ডিস্ক, ভল্ট সেটিং ও রিকভারি কী, ইঞ্জিনের তথ্য (শুধু দেখা), প্রাইভেসি। আলাদা ইঞ্জিন-স্ক্রিন, প্যানিক-স্ক্রিন আর রিসোর্স মনিটর বাদ।

৪. আর্কিটেকচার

```
Vanta.exe  (Tauri 2 · WebView2 · UI: React + TypeScript)
 └─ src-tauri: commands, tray, Ctrl+L
     └─ vanta-core
         ├─ vanta-types        shared types, exit codes 86/87/88              [done, tested]
         ├─ vanta-vault        Argon2id, per-profile keys, recovery, auto-lock  [done, tested]
         ├─ vanta-store        SQLite + FTS5, search language                 [done, tested]
         ├─ vanta-fingerprint  device DB, UA/UA-CH, rules C001-C030, signing    [done, tested]
         ├─ vanta-proxy        local SOCKS5 relay, kill switch, DoH, QUIC block, 6 provider grammars
         ├─ vanta-launcher     spawn engine in a Job Object (kill on close), no CDP
         ├─ vanta-health       56 checks, TLS/HTTP2 reflector, STUN, probe page
         └─ vanta-log          encrypted hash-chained activity log
 engine:  %LOCALAPPDATA%\Programs\Vanta\engine\vanta-chromium.exe  (one version, ships with the app)
 data:    %LOCALAPPDATA%\Vanta\      install: %LOCALAPPDATA%\Programs\Vanta (no admin)
```

স্তর	প্রযুক্তি	কেন
ম্যানেজার অ্যাপ	Rust + Tauri 2, Windows-এর নিজের WebView2	আলাদা ব্রাউজার বয়ে বেড়ায় না; বসে থাকলে RAM কম, CPU প্রায় ০%
UI	React + TypeScript, এই নথির ডিজাইন-সিস্টেম	১০টা স্ক্রিন, ছবির রং/মাপ সরাসরি কোডে
ডাটা	SQLite + FTS5	১০,০০০ প্রোফাইলে মিলিসেকেন্ডে সার্চ
ক্রিপ্টো	Argon2id, XChaCha20-Poly1305, Ed25519, minisign	পরীক্ষিত, আধুনিক
প্রক্সি TLS	rustls (ring)	OpenSSL লাগে না
ইঞ্জিন	ungoogled-chromium 156 + Vanta প্যাচ (C++)	ফিঙ্গারপ্রিন্ট ইঞ্জিনের ভেতরে

ইতিমধ্যে যা বানানো হয়েছে: প্রথম চারটা ক্রেট (types, vault, store, fingerprint) Rust-এ লেখা এবং এই পরিকল্পনা অনুযায়ী আপডেট করা (Android, প্যানিক, এজিং সরানো; নিয়ম C001–C030)। সব টেস্ট পাস, clippy ও fmt পরিষ্কার, Windows টার্গেটে কমপাইল হয়। Chrome 156-এর GREASE brand ও ক্রম আসল Chromium সোর্স থেকে পোর্ট করা, আর আসল Chrome 131-এর হেডারের সাথে মিলিয়ে যাচাই করা।

৫. ইঞ্জিন পরিকল্পনা

৫.১ স্তর

1. Chromium সোর্স
2. ungoogled-chromium প্যাচ (Google সার্ভিস ও টেলিমেট্রি বাদ)
3. ungoogled-chromium-windows (Windows প্যাচ, build.py, package.py, পরপর-চলা Actions জব)
4. **Vanta প্যাচ**: `patches/vanta/NNNN-*.patch` + `patches/series`

৫.২ রিপোজিটরি

- আপনার GitHub-এ ungoogled-chromium-windows fork → `vanta-engine`, ট্যাগ 156.0.8078.79-1.1।
- প্রতি push-এ দ্বিতীয় জায়গায় মিরর (Codeberg বা GitLab)।
- নতুন ফাইল: `vanta/VERSION` (R5-এর একমাত্র উৎস), `vanta/flags.vanta.gn`, `.github/workflows/vanta-*.yml`।

৫.৩ প্যাচ দল (৮টা; প্রতিটা S1–S3 পাস করলে তবেই পরেরটা)

দল	কী বদলায়	উৎস
G1	নাম (vanta-chromium.exe), ব্র্যান্ডিং, সাইন করা কনফিগ পড়া, Ed25519 যাচাই (ভুল হলে exit 86), ভার্সন-লক (অমিল হলে exit 87)	নতুন
G2	navigator, User-Agent, UA-CH, Sec-CH-UA ("Not:A-Brand";v="8", "Chromium";v="156", "Google Chrome";v="156")	Vanta 0001/0002 পোর্ট
G3	স্ক্রিন, hardwareConcurrency, deviceMemory, maxTouchPoints (শুধু ডেস্কটপ)	Vanta 0003 পোর্ট
G4	Canvas 2D, WebGL (vendor/renderer/প্যারামিটার), WebGPU — Real/Noise/Block	Vanta 0004 পোর্ট (156-এ সংঘর্ষ মেটানো)
G5	AudioContext / OfflineAudioContext noise	Vanta 0004/0005
G6	ফন্ট তালিকা, ClientRects/টেক্সট মাপ	নতুন, fingerprint-chromium পড়ে শেখা
G7	টাইমজোন, লোকেল, Accept-Language, জিওলোকেশন, স্পিচ ভয়েস, মিডিয়া ডিভাইস	নতুন
G8	WebRTC disable_non_proxied_udp বাধ্যতামূলক, QUIC/IPv6/LAN গেট, ব্যাকগ্রাউন্ড নেটওয়ার্ক বন্ধ	Vanta 0006 পোর্ট

v2-এর G9 (অটোমেশন লুকানো, CDP screenX বাগ) বাদ, কারণ অটোমেশন আর নেই এবং লঞ্চার CDP ব্যবহার করে না।

৫.৪ দ্রুত-ব্যর্থ পাইপলাইন (R7)

ধাপ	কী	সময় (আনুমানিক)	রানার
S1	সব প্যাচ পরপর বসে কিনা, কমপাইল ছাড়া	~১০ মিনিট	ubuntu
S2	gn gen --check, Vanta ফ্ল্যাগসহ	~৩০ মিনিট	windows

ধাপ	কী	সময় (আনুমানিক)	রানার
S3	শুধু Vanta যে টার্গেট ছোঁয় সেগুলো কমপাইল	~১–২ ঘণ্টা	windows
S4	পুরো build, পরপর জব, artifact থেকে চালু	~১.৫ দিন (যাচাই বাকি)	windows
S5	স্মোক টেস্ট: exe চালু, ফিঙ্গারপ্রিন্ট = কনফিগ, শুধু-প্রক্সি ট্রাফিক, কিল সুইচ	~২০ মিনিট	windows
S6	SHA-256, minisign, (ঐচ্ছিক) Authenticode, GitHub Release	~১০ মিনিট	windows

নিয়ম: S1–S3 পাস না করলে S4 শুরু হয় না; S5 পাস না করলে রিলিজ নয়। ফ্রি রানার: ৪ vCPU, ১৬ GB RAM, ১৪ GB SSD, প্রতি জব ৬ ঘণ্টা।

৬. অ্যাপ ↔ ইঞ্জিন চুক্তি

- **খোঁজা:** `<install>\engine\vanta-chromium.exe` (একটাই ইঞ্জিন, অ্যাপের সাথে আসে)।
- **লঞ্চ:** ইনস্টল করা ইঞ্জিনের ভার্সন পড়া → পারসোনা সেই ভার্সনে বাঁধা → নিয়ম C001–C030 → কনফিগ সাইন (Ed25519) → লোকাল রিলে 127.0.0.1 + DoH → ইঞ্জিন Job Object-এর ভেতরে (KILL_ON_JOB_CLOSE)। কোনো ডিবাগ-পোর্ট বা CDP নয়। প্রক্সির পাসওয়ার্ড কখনো ইঞ্জিনের আর্গুমেন্টে যায় না।
- **ভার্সন-লক (R5):** অ্যাপের দিকে C030, ইঞ্জিনের দিকে G1 প্যাচ; দুই স্তর।
- **এক্সিট কোড:** 86 = কনফিগ সাইন ভুল, 87 = ভার্সন অমিল, 88 = আইসোলেশন ভাঙা (কোডে ইতিমধ্যে সংজ্ঞায়িত ও টেস্ট করা)।

৭. রিলিজ ও অ্যাকাউন্ট-হারানো সুরক্ষা (R8)

- একটাই ইনস্টলার: অ্যাপ + ইঞ্জিন একসাথে। নতুন ভার্সন = নতুন ইনস্টলার চালানো; অ্যাপের ভেতরে আপডেটার নেই, তাই আলাদা আপডেট-সার্ভার বা ডোমেইনও লাগে না।
- রিলিজ-কী আগে অফলাইনে: minisign private key + পাসওয়ার্ড (আর ঐচ্ছিক Authenticode PFX)। দুটো এনক্রিপ্টেড USB কপি, তারপর GitHub Secrets-এ।
- প্রতি রিলিজের ইনস্টলার + SHA-256 এক্সটার্নাল ড্রাইভে।
- Git: লোকাল clone + দ্বিতীয় রিমোট মিরর; GitHub-এ 2FA + প্রিন্ট করা রিকভারি কোড।
- build স্ক্রিপ্ট যেকোনো জায়গায় চলে (নিজের PC-তে build.ps1)।

৮. পারফরম্যান্স বাজেট (R6)

মাপ	লক্ষ্য
অ্যাপ চালু থেকে লক স্ক্রিন	< ১ সেকেন্ড
আনলক	০.৫–১ সেকেন্ড (ইচ্ছে করে, Argon2id)
ম্যানেজার বসে থাকলে RAM	< ৮০ MB (v2-তে ছিল ১০০)
ম্যানেজার বসে থাকলে CPU	~০%
প্রোফাইল লঞ্চ	< ২ সেকেন্ড
বন্ধ প্রোফাইল	০ RAM, ০ CPU
১০,০০০ প্রোফাইল সার্চ	< ৫০ ms
ইনস্টলার	একটা ফাইল, অ্যাডমিন ছাড়া

থাকছে: Memory Saver, renderer সীমা, GPU টগল, ক্যাশ পরিষ্কার, আর CI-তে গতি-পরীক্ষা যা বাজেট পেরোলে build ফেইল করে। বাদ: অটো-স্লিপ, প্রতি-প্রোফাইল RAM/CPU সীমা, ট্রাফিক মিটার (F17-এর সাথে)। সংখ্যাগুলো লক্ষ্য; আসল মাপ হবে আপনার Windows PC-তে (P6)।

৯. মান-পরীক্ষা (প্রতি ধাপে)

- `scripts/check.sh` : `cargo fmt --check` , `cargo clippy -D warnings` , `cargo test --workspace` , Windows টার্গেটে `cargo check` , `no-telemetry`
- `pnpm build` , `pnpm test` , আর UI স্ক্রিনশট-তুলনা এই নথির ১০টা ছবির সাথে
- টেস্ট ছাড়া কোনো ফিচার merge হবে না; S5 ছাড়া কোনো ইঞ্জিন রিলিজ নয়
- বাইরের বেঞ্চমার্ক (শুধু মাপার জন্য): CreepJS, BrowserLeaks, আর আমাদের হেলথ চেক

১০. কাজের ক্রম

ধাপ	কাজ	যা লাগবে	অবস্থা
P0	আপনি GitHub কানেক্ট করবেন (repo + workflow অনুমতি); মিরর হোস্ট বেছে নেবেন	আপনি	বাকি
P1	vanta-engine fork + মিরর; S1/S2 ওয়ার্কফ্লো; প্যাচ দল G1	GitHub	P0-এর অপেক্ষায়
P2	Rust ক্রেট: types, vault, store, fingerprint	স্যান্ডবক্স	**শেষ, টেস্ট পাস**
P2b	vanta-proxy, vanta-launcher, vanta-log; Tauri শেল + ১০টা স্ক্রিনের UI	স্যান্ডবক্স	পরের কাজ
P3	প্যাচ দল G2–G8; S3 সবুজ	GitHub Actions	P1-এর পরে
P4	F03 হেলথ চেক + বাকি ফিচারের "শেষ" পরীক্ষা	স্যান্ডবক্স	—
P5	পুরো ইঞ্জিন build S4 → S5 → প্রথম সাইন করা ইঞ্জিন	Actions + কী	—
P6	ইনস্টলার (অ্যাডমিন ছাড়া) + আপনার Windows PC-তে আসল টেস্ট ও গতি-মাপ	আপনার PC	—
P7	পাওয়া সমস্যা ঠিক করা; v1.0	—	—

১১. আপনার সিদ্ধান্ত লাগবে

#	প্রশ্ন	আমার পরামর্শ
D1	পণ্যের নাম: Vanta নাকি Veil?	ডিজাইন ও কোডে এখন Vanta
D2	GitHub কানেক্ট (P0)	ইঞ্জিন বানানো শুরুর জন্য দরকার
D3	মিরর হোস্ট: Codeberg নাকি GitLab	Codeberg
D4	Authenticode সাইনিং সার্টিফিকেট (টাকা লাগে)	প্রথমে শুধু minisign; তখন Windows SmartScreen সতর্কতা দেখাবে
D5	প্রোপ্রাইটারি কোডেক (H.264/AAC)	ডিফল্ট বন্ধ; Widevine নেই

v2-এর "আপডেটের জন্য নিজের ডোমেইন" প্রশ্নটা আর লাগে না (অ্যাপের ভেতরে আপডেটার নেই)। rustls + ring সিদ্ধান্ত ধরে নেওয়া হলো।

১২. জানা ঝুঁকি

- **আপডেট হাতে:** অ্যাপের ভেতরে আপডেটার না থাকায় Chromium-এর নিরাপত্তা-আপডেট পেতে আপনাকে নতুন ইনস্টলার চালাতে হবে। চাইলে পরে শুধু "নতুন ভার্সন আছে" জানানোর ছোট একটা চেক যোগ করা যায়।
- **ব্যাকআপ নেই:** PC নষ্ট হলে প্রোফাইল হারাবে, যদি না আপনি %LOCALAPPDATA%\Vanta ফোল্ডারটা নিজে কোথাও কপি রাখেন।
- ইঞ্জিন Chrome Stable-এর ১–২ ভার্সন পেছনে থাকবে; গ্রহণযোগ্য, R5 সঙ্গতি রাখে।

- পুরো build ফ্রি রানারে ~১.৫ দিন (দাবি, যাচাই বাকি)।
- WebGL প্যারামিটার হোস্টের GPU থেকে আসে, তাই প্রোফাইলের GPU পরিবার হোস্টের সাথে মেলা ভালো।
- ডিভাইস ডাটাবেস ছোট (১৪টা ডেস্কটপ রেকর্ড)।
- এই স্যান্ডবক্সে Windows অ্যাপ আর পুরো ইঞ্জিন চালানো যায় না; P6-ই প্রথম আসল প্রমাণ।